Data Flow Diagram

Phase 2 | Request, generation, persistence, and display

Context-level flow

```
Traveler
| trip preferences + account credentials
v
React frontend (Vite)
| JSON requests; bearer token for protected routes
v
FastAPI backend
|-- auth service -> Supabase users / in-memory development store
|-- AI orchestrator -> Groq or Ollama -> structured itinerary
| `-> local curated/procedural fallback
|-- trip services -> Supabase trips + itinerary days / in-memory store
`-- history service -> Supabase history / in-memory log
^ JSON responses
React renders itinerary and caches selected saved-trip data in browser local storage
```

Main itinerary flow

Step	Data movement
1. Capture	Trip form collects destination, dates, traveler count, budget/currency, style, pace, interests, and dietary needs.
2. Authenticate	Frontend sends access token in Authorization header.
3. Validate	FastAPI validates request fields against Pydantic schemas.
4. Generate	AI orchestrator builds prompt; uses configured Groq/Ollama or local fallback.
5. Return	API serializes response with trip and itinerary-day data.
6. Persist	Save action sends itinerary to create endpoint; trip and day services store it.
7. Render	Frontend updates trip detail and local per-user cache; backend remains source of truth when configured.

Trust and privacy boundaries

- Passwords are hashed before storage; access tokens are signed by the backend.
- The Supabase service-role key is backend-only.
- Trip endpoints receive the user identity from the verified token and enforce ownership in service operations.
- The browser cache is user-scoped by local user ID but remains local device storage.

Failure paths

Remote model errors can route itinerary generation to local logic. Supabase errors may be surfaced to the caller or use the development fallback depending on service and operation. Local memory and browser cache are not durable cross-device substitutes for a working database.

Prepared from the current project source and configuration. Product assumptions and proposed work are labeled; no user research, test result, or deployment result is implied unless explicitly stated.